

Lecture -4 (Inheritance)

- we had the point class. Now I want colourpoint class with all the functionality of point along with store colour.
- Inheritance - create a new class based on an existing class, automatically obtaining its attributes & methods

```

class ColouredPoint(Point):
    def __init__(self, x, y, color):
        super().__init__(x, y)
        self.color = color
  
```

inherits the methods from point class. super() gives access to parent class.

Eg. class Vehicle:

```

def __init__(self, brand):
    self.brand = brand
def start(self):
    print("start")
  
```

class Car(Vehicle):

```

def __init__(self, brand, doors):
    super().__init__(brand)
    self.doors = doors
  
```

- why inheritance? → suppose every employee has a name & salary. But different types of employees may have different responsibilities

$$I = \int_a^b f(x) dx$$

class integrator:

```

def __init__(self, f, a, b, n):
    self.f = f
    self.a = a
    self.b = b
    self.n = n
  
```

numerical integration

↳ Trapezoidal Rule, Simpson's Rule

$$Eg. \quad I = \int_0^2 x^2 dx$$

f = lambda x: x**2

a=0, b=2, n=4

no. of sub intervals

- Trapezoidal Rule

$[a, b]$ in n piece width $h = \frac{b-a}{n}$
 instead of approx. the area using n rectangles, the trapezoidal rule approximate each sectⁿ using a trapezoid.

$$\int_a^b f(x) dx \approx h \left[\frac{f(x_0)}{2} + \underbrace{f(x_1) + f(x_2) + \dots + f(x_{n-1})}_{\text{all pts. in blue have wt. 1.}} + \frac{f(x_n)}{2} \right]$$

all pts. in blue have wt. 1.

first & last pt. have wt. $\frac{1}{2}$

$$Eg. \quad \int_0^2 x^2 dx \xrightarrow{\text{actual}} \frac{x^3}{3} \rightarrow \frac{8}{3} = 2.667$$

↓ trapezoidal

n=4, h=0.5

→ 0 0.5 1 1.5 2

funcⁿ 0 0.25 1 2.25 4

$$T = \frac{1}{2} \left(\frac{0}{2} + 0.25 + 1 + 2.25 + \frac{4}{2} \right)$$

$$= 0.5 (0 + 3.5 + 2) = 0.5 \times 5.5 = 2.75$$

- Simpson's Rule → more accurate, since instead of approximating the curve using straight lines, it approximates pieces of curve using quadratic curves / parabola.

n must be even

$$h = \frac{b-a}{n}$$

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4f(x_1) + 2f(x_2) + \dots + 4f(x_{n-1}) + f(x_n) \right]$$

first & last point $\rightarrow 1$
 odd points $\rightarrow 4$
 even pts. $\rightarrow 2$

} coefficients

$$\int_0^2 x^2 dx$$

$$h = \frac{2}{4} = \frac{1}{2} = 0.5$$

$$\int_0^2 f(x) dx = \frac{0.5}{3} (0 + 4(0.25) + 2(1) + 4(2.25))$$

$$= \frac{0.5 \times 16}{3} = \frac{8}{3}$$

class T-Integrator (Integrator):

def integrate(self):

h = (self.b - self.a) / self.n

total = 0.5 * self.f(self.a)

total += 0.5 * self.f(self.b)

for i in range(1, self.n):

x = self.a + i * h

total += self.f(x)

return h * total